



LoraTrack

Professional Deployment and Operations Guide

Document version: 1.0

Classification: Public product documentation

Document Control

Field	Value
Product	LoraTrack
Document type	Professional Deployment and Operations Guide
Document version	1.0
Audience	Users, administrators, engineering, operations, and security teams

This documentation describes product capabilities and procedures. References to practices or standards do not constitute certification, independent assurance, or formal customer acceptance.

Document Index

- [Dependency Matrix](#)
- [Deployment and Environments](#)
- [Ubuntu LTS Deployment Tutorial](#)
- [Windows Server and IIS Deployment Tutorial](#)
- [Recommended SQL Server Baseline](#)
- [Compliance Baseline and Benchmarks](#)
- [Operations, Monitoring, and Runbooks](#)
- [Field Commissioning Guide](#)

Dependency Matrix

This matrix reflects the dependencies observed in `composer.json` and the installed direct package versions reported by `composer show --direct`.

Runtime

Component	Required or Recommended Version	Notes
PHP	8.2 or higher	Laravel 12 requires a modern PHP runtime. PHP 8.3/8.4 may be used if extensions are available and validated.
Microsoft SQL Server	2022 Standard or 2025 Standard	Recommended for Windows/IIS enterprise deployments under the ISO/CIS baseline. Requires <code>pdo_sqlsrv</code> and Microsoft ODBC Driver for SQL Server.
MariaDB	10.6 or higher	Technical alternative supported by Laravel and used as the original project reference.
MySQL	8.0 or higher	Technical alternative supported by Laravel.
Composer	2.x	PHP dependency manager.
Linux web server	Nginx or Apache	The Ubuntu tutorial uses Nginx + PHP-FPM.
Windows web server	IIS 10	Requires FastCGI and URL Rewrite.
Queue	Laravel Queue	Database queue or another configured driver.
Scheduler	cron, systemd timer, Task Scheduler, or supervisor	Required for <code>schedule:run</code> and workers.

Direct PHP Packages

Package	Installed Version	Usage
<code>laravel/framework</code>	12.62.0	Main framework.
<code>laravel/socialite</code>	5.28.0	OAuth/OIDC login.
<code>socialiteproviders/microsoft</code>	4.9.0	Microsoft provider.
<code>laravel/tinker</code>	2.11.1	Interactive console.
<code>php-mqtt/client</code>	2.3.0	MQTT listener.
<code>laravel/pint</code>	1.29.3	PHP code formatting.
<code>phpunit/phpunit</code>	11.5.55	Automated tests.

PHP Extensions

Install and enable:

- `bcmath`
- `ctype`
- `curl`
- `dom`
- `fileinfo`
- `gd`
- `json`
- `mbstring`
- `openssl`
- `pdo`
- `pdo_mysql` when using MySQL/MariaDB
- `pdo_sqlsrv` when using SQL Server
- `sqlsrv` recommended for diagnostics and non-PDO tooling
- `tokenizer`
- `xml`
- `xmlreader`
- `xmlwriter`
- `zip`

Recommended:

- `opcache`

- `intl`
- `redis` if Redis is used for cache, queues, or sessions.

Optional External Services

Service	Usage
Microsoft SQL Server	Recommended database for Windows/IIS deployments under the ISO/CIS baseline.
The Things Industries	LoRaWAN webhook source.
MQTT broker	Generic telemetry or TTI through MQTT.
Meraki	Location API.
SAP S/4HANA	Product Master catalog.
Microsoft Entra ID	Microsoft login.
SMTP	Email alerts and invitations.

Relevant Configuration Files

- `.env`
- `config/app.php`
- `config/database.php`
- `config/queue.php`
- `config/mail.php`
- `config/filesystems.php`
- `routes/api.php`
- `routes/console.php`

Deployment and Environments

Base Requirements

- PHP 8.2 or higher.
- Composer.
- SQL Server 2022/2025, MariaDB 10.6+, or MySQL 8+ depending on deployment baseline.
- PHP extensions required by Laravel and the selected database.
- Cron, Task Scheduler, Supervisor, systemd, or equivalent process management.
- TLS on the public domain.

Environment Variables

Start from `.env.example`. Do not generate `.env` from a pipeline containing embedded secrets.

Critical variables:

- `APP_ENV`
- `APP_KEY`
- `APP_DEBUG=false` in production
- `APP_URL`
- `DB_*`
- `QUEUE_CONNECTION`
- `MYSQL_ATTR_SSL_CA` for MySQL/MariaDB deployments that require TLS.
- `MYSQL_ATTR_MAX_BUFFER_SIZE=6291456` when PDO uses `libmysql`, so Meraki payloads up to the 5 MiB HTTP limit are not truncated while being read.
- `CACHE_STORE`
- `SESSION_DRIVER`
- `MAIL_*`
- `MICROSOFT_*` when Microsoft Entra ID is used

Initial Installation

```
composer install --no-dev --optimize-autoloader
php artisan key:generate
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
```

Do not regenerate `APP_KEY` in an existing production environment because it invalidates encrypted data and sessions.

File Permissions

Laravel requires write access to:

- `storage/`
- `bootstrap/cache/`

Floor plans must remain in private storage:

```
storage/app/private
```

Do not expose floor plans through a public symlink.

Scheduler

Run every minute:

```
php artisan schedule:run
```

Scheduled tasks:

- `loratrack:evaluate-alerts` every ten minutes.
- `loratrack:manage-telemetry-storage` hourly.
- `loratrack:prune-meraki-history` hourly.

Scheduler

The Laravel scheduler must run every minute. It drains the durable webhook inboxes and processes observations, TTI/MQTT events, and requested catalog synchronizations. Laravel Queue is not required. Avoid duplicate scheduler cron entries when the database has low connection limits.

Recommended Environments

Environment	Purpose	Data
local	development	synthetic data
test/ci	automated tests	ephemeral database
staging	customer validation	anonymized or approved data
production	live operation	controlled data

Do not use real payloads or real floor plans in non-production without approval and controls.

Backup and Restore

The plan must cover:

- database;
- `storage/app/private`;
- production `.env`;
- logs required by audit;
- deployed release version.

Initial frequency recommendation:

- daily full backup;
- retention based on contract;
- restore test quarterly or before critical releases.

Minimum Monitoring

- HTTP availability;
- 5xx errors;
- failed jobs;
- queue depth;
- database growth;
- disk space;
- inactive connectors;
- telemetry pending beyond threshold;
- TLS certificates;
- token and credential expiration.

Rollback

Define before each change:

- previous release version;
- migration compatibility;
- pre-deploy backup;
- worker pause procedure;
- job retry procedure;
- user communication plan.

Production Hardening

- `APP_DEBUG=false`.
- Mandatory TLS.
- Database not publicly exposed.
- Least-privilege database users.
- Encrypted backups.
- Secret rotation.
- WAF or reverse proxy when applicable.
- Request size limits.
- Centralized logs.
- Disable test accounts.

Ubuntu LTS Deployment Tutorial

Scope

Tutorial for deploying LoraTrack on Ubuntu Server LTS with Nginx, PHP-FPM, Composer, and cron. The recommended Microsoft database backend is SQL Server 2022/2025 on a dedicated server, certified Linux host, Windows Server, or Azure SQL service under the ISO/CIS baseline.

Project versions:

- Laravel Framework 12.62.0.
- Required PHP: 8.2+.
- Recommended database: SQL Server 2022 Standard with the latest approved CU, or SQL Server 2025 Standard after staging validation.
- MQTT client: `php-mqtt/client` 2.3.0.
- Microsoft OAuth: `laravel/socialite` 5.28.0 and `socialiteproviders/microsoft` 4.9.0.

1. Prepare the Server

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y software-properties-common unzip git curl ca-certificates gnupg
```

2. Install PHP and Extensions

```
sudo apt install -y php php-fpm php-cli php-common php-mbstring php-xml php-curl php-zip
php-bcmath php-gd php-intl php-opcache
```

Verify:

```
php -v
php -m | grep -E "bcmath|curl|fileinfo|gd|mbstring|openssl|xml|zip"
```

Suggested `php.ini` values:

```
memory_limit=256M
upload_max_filesize=25M
post_max_size=30M
max_execution_time=120
date.timezone=America/Santiago
opcache.enable=1
opcache.enable_cli=1
```

Restart FPM:

```
sudo systemctl restart php*-fpm
```

3. Install Nginx

```
sudo apt install -y nginx
sudo systemctl enable nginx
sudo systemctl start nginx
```

4. Configure SQL Server Connectivity

Recommended enterprise baseline:

- SQL Server 2022 Standard with latest approved CU for conservative production.
- SQL Server 2025 Standard for controlled new adoption after staging validation.
- SQL Server Developer for development.
- SQL Server Express only for labs or small pilots with explicit limit acceptance.

Install Microsoft ODBC Driver for SQL Server and PHP `sqlsrv` / `pdo_sqlsrv` extensions following Microsoft's repository instructions for the Ubuntu version.

Validate:

```
php -m | grep -E "sqlsrv|pdo_sqlsrv"
sudo systemctl restart php*-fpm
```

Database account example:

```

create database loratrack;
go
create login loratrack_app with password = 'CHANGE_ME_LONG_SECRET';
go
use loratrack;
go
create user loratrack_app for login loratrack_app;
go
alter role db_datareader add member loratrack_app;
alter role db_datawriter add member loratrack_app;
alter role db_ddladmin add member loratrack_app;
go

```

After migrations, evaluate removing DDL permissions from the runtime account and using a separate migration account.

5. Install Composer

```

cd /tmp
php -r "copy('https://getcomposer.org/installer', 'composer-setup.php');"
php composer-setup.php
sudo mv composer.phar /usr/local/bin/composer
composer --version

```

Validate the official checksum before running the installer in controlled environments.

6. Application Directory

```

sudo adduser --system --group --home /var/www/loratrack loratrack
sudo mkdir -p /var/www/loratrack/current
sudo chown -R loratrack:www-data /var/www/loratrack
sudo -u loratrack git clone REPO_URL /var/www/loratrack/current
cd /var/www/loratrack/current

```

Install dependencies:

```

sudo -u loratrack composer install --no-dev --optimize-autoloader

```

7. Configure `.env`

```
sudo -u loratrack cp .env.example .env
sudo -u loratrack nano .env
```

Minimum values:

```
APP_NAME=LoraTrack
APP_ENV=production
APP_DEBUG=false
APP_URL=https://loratrack.example.com
APP_TIMEZONE=America/Santiago

DB_CONNECTION=sqlsrv
DB_HOST=sqlserver.example.com
DB_PORT=1433
DB_DATABASE=loratrack
DB_USERNAME=loratrack_app
DB_PASSWORD=CHANGE_ME_LONG_SECRET

QUEUE_CONNECTION=database
CACHE_STORE=database
SESSION_DRIVER=database
```

Generate `APP_KEY` only for a new installation:

```
sudo -u loratrack php artisan key:generate
```

8. Permissions, Migrations, and Cache

```
sudo chown -R loratrack:www-data /var/www/loratrack/current
sudo find /var/www/loratrack/current -type f -exec chmod 0644 {} \;
sudo find /var/www/loratrack/current -type d -exec chmod 0755 {} \;
sudo chmod -R ug+rwX /var/www/loratrack/current/storage
/var/www/loratrack/current/bootstrap/cache

sudo -u loratrack php artisan migrate --force
sudo -u loratrack php artisan config:cache
sudo -u loratrack php artisan route:cache
sudo -u loratrack php artisan view:cache
```

9. Nginx Site

Use `/var/www/loratrack/current/public` as the web root.

```
server {
    listen 80;
    server_name loratrack.example.com;
    root /var/www/loratrack/current/public;
    index index.php index.html;
    client_max_body_size 30M;

    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ /\.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~ /\.(!well-known).* {
        deny all;
    }
}
```

Adjust the PHP-FPM socket to the installed version.

10. TLS

Use an approved PKI certificate or Let's Encrypt where allowed.

11. Scheduler

Scheduler cron:

```
* * * * * cd /var/www/loratrack/current && php artisan schedule:run >>
storage/logs/schedule.log 2>&1
```

No Laravel Queue systemd service is required.

12. Optional Microsoft Login

```
MICROSOFT_CLIENT_ID=  
MICROSOFT_CLIENT_SECRET=  
MICROSOFT_TENANT_ID=  
MICROSOFT_REDIRECT_URI=https://loratrack.example.com/auth/microsoft/callback
```

Then run:

```
sudo -u loratrack php artisan config:cache
```

13. Post-Deployment Verification

```
curl -I https://loratrack.example.com/login  
sudo -u loratrack php artisan about  
sudo -u loratrack php artisan schedule:run
```

Validate login, dashboard, `/operations/health`, connector creation, TTI ingestion, private floor plan access, and scheduled processing.

Windows Server and IIS Deployment Tutorial

Scope

Tutorial for deploying LoraTrack on Windows Server with IIS, PHP FastCGI, Composer, Microsoft SQL Server, and Task Scheduler.

Project versions:

- Laravel Framework 12.62.0.
- Required PHP: 8.2+.
- Recommended database: Microsoft SQL Server 2022 Standard with latest approved CU, or SQL Server 2025 Standard after ISO/CIS staging validation.
- Microsoft OAuth: `laravel/socialite` 5.28.0 and `socialiteproviders/microsoft` 4.9.0.
- MQTT client: `php-mqtt/client` 2.3.0.

1. Required Components

Install:

- Windows Server 2019/2022/2025.
- IIS 10.
- PHP 8.2+ Non Thread Safe x64.
- Visual C++ Redistributable required by PHP.
- Composer 2.x for Windows.
- Microsoft SQL Server 2022/2025.
- Microsoft ODBC Driver for SQL Server.
- Microsoft Drivers for PHP for SQL Server: `pdo_sqlsrv` and `sqlsrv`.
- IIS URL Rewrite Module.
- Git for Windows or an approved deployment mechanism.

2. Enable IIS and CGI

PowerShell as Administrator:

```
Install-WindowsFeature Web-Server, Web-CGI, Web-Common-Http, Web-Default-Doc, Web-Static-Content, Web-Http-Errors, Web-Http-Redirect, Web-Filtering, Web-Mgmt-Console
```

Install URL Rewrite from an approved internal package or Microsoft source.

3. Install PHP

Install PHP NTS x64, for example:

```
C:\PHP\8.3
```

Copy `php.ini-production` to `php.ini` and enable:

```
extension_dir="ext"
extension=bcmath
extension=curl
extension=fileinfo
extension=gd
extension=mbstring
extension=openssl
extension=pdo_sqlsrv
extension=sqlsrv
extension=zip

cgi.force_redirect=0
cgi.fix_pathinfo=1
fastcgi.impersonate=1

memory_limit=256M
upload_max_filesize=25M
post_max_size=30M
max_execution_time=120
date.timezone=America/Santiago
opcache.enable=1
opcache.enable_cli=1
```

Verify:

```
php -v
php -m
```

4. Configure PHP FastCGI in IIS

Add a handler mapping:

- Request path: `*.php`
- Module: `FastCgiModule`
- Executable: `C:\PHP\8.3\php-cgi.exe`

- Name: PHP via FastCGI

5. SQL Server Setup

Recommendation:

- Conservative production: SQL Server 2022 Standard, latest approved CU.
- Controlled new adoption: SQL Server 2025 Standard, validated in staging.
- Development: SQL Server Developer.
- Lab: SQL Server Express only if limits are accepted.

Create database and login:

```
create database loratrack;
go
create login loratrack_app with password = 'CHANGE_ME_LONG_SECRET';
go
use loratrack;
go
create user loratrack_app for login loratrack_app;
go
alter role db_datareader add member loratrack_app;
alter role db_datawriter add member loratrack_app;
alter role db_ddladmin add member loratrack_app;
go
```

Validate:

```
select @@version;
```

6. Application Directory

```
New-Item -ItemType Directory -Force C:\inetpub\loratrack
git clone REPO_URL C:\inetpub\loratrack
cd C:\inetpub\loratrack
composer install --no-dev --optimize-autoloader
```

7. Configure `.env`

```
Copy-Item .env.example .env
notepad .env
```

Minimum values:

```
APP_NAME=LoraTrack
APP_ENV=production
APP_DEBUG=false
APP_URL=https://loratrack.example.com
APP_TIMEZONE=America/Santiago

DB_CONNECTION=sqlsrv
DB_HOST=127.0.0.1
DB_PORT=1433
DB_DATABASE=loratrack
DB_USERNAME=loratrack_app
DB_PASSWORD=CHANGE_ME_LONG_SECRET

QUEUE_CONNECTION=database
CACHE_STORE=database
SESSION_DRIVER=database
```

Generate `APP_KEY` only for a new installation:

```
php artisan key:generate
```

8. Configure IIS Site

The site physical path must be:

```
C:\inetpub\loratrack\public
```

Example:

```
Import-Module WebAdministration
New-Website -Name "LoraTrack" -Port 80 -HostHeader "loratrack.example.com" -PhysicalPath
"C:\inetpub\loratrack\public"
```

Configure HTTPS using an approved PKI or public CA certificate.

9. web.config

Create `C:\inetpub\loratrack\public\web.config`:

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <rule name="Laravel" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true" />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory" negate="true" />
          </conditions>
          <action type="Rewrite" url="index.php" />
        </rule>
      </rules>
    </rewrite>
    <security>
      <requestFiltering>
        <requestLimits maxAllowedContentLength="31457280" />
      </requestFiltering>
    </security>
    <defaultDocument>
      <files>
        <clear />
        <add value="index.php" />
      </files>
    </defaultDocument>
  </system.webServer>
</configuration>
```

Do not point IIS to the project root.

10. NTFS Permissions

Grant read access to the project and write access only to:

- `storage`
- `bootstrap\cache`

Example:

```
icacls C:\inetpub\loratrack /grant "IIS AppPool\LoraTrack:(OI)(CI)RX"  
icacls C:\inetpub\loratrack\storage /grant "IIS AppPool\LoraTrack:(OI)(CI)M"  
icacls C:\inetpub\loratrack\bootstrap\cache /grant "IIS AppPool\LoraTrack:(OI)(CI)M"
```

11. Migrate and Optimize

```
php artisan migrate --force  
php artisan config:cache  
php artisan route:cache  
php artisan view:cache
```

12. Task Scheduler

Scheduler task:

- Program: `C:\PHP\8.3\php.exe`
- Arguments: `artisan schedule:run`
- Start in: `C:\inetpub\loratrack`
- Repeat every minute.

No Laravel Queue task or persistent queue service is required.

13. Optional Microsoft Login

```
MICROSOFT_CLIENT_ID=  
MICROSOFT_CLIENT_SECRET=  
MICROSOFT_TENANT_ID=  
MICROSOFT_REDIRECT_URI=https://loratrack.example.com/auth/microsoft/callback
```

Apply:

```
php artisan config:cache  
iisreset
```

14. Post-Deployment Verification

```
Invoke-WebRequest https://loratrack.example.com/login -UseBasicParsing  
php artisan about  
php artisan schedule:run
```

Validate login, dashboard, `/operations/health`, floor plan access, connector creation, telemetry processing, and email delivery.

15. Troubleshooting

- Error 500: inspect `storage\logs\laravel.log` and Windows Event Viewer.
- Laravel routes return 404: verify URL Rewrite and `web.config`.
- Permission errors: reapply permissions on `storage` and `bootstrap\cache`.
- `.env` changes do not apply: run `php artisan config:clear`, `php artisan config:cache`, and `iisreset`.
- Telemetry does not process: run `php artisan schedule:run -v` manually and inspect Task Scheduler history.

Recommended SQL Server Baseline

Recommendation

For regulated or high-criticality industrial customers, the database recommendation is fixed against the [compliance baseline](#):

- **Conservative production:** Microsoft SQL Server 2022 Standard, updated to the latest approved Cumulative Update.
- **Controlled new production adoption:** Microsoft SQL Server 2025 Standard, only after staging validates migrations, PHP drivers, CIS hardening, and expected LoraTrack load.
- **Development:** SQL Server Developer using the same major version as production.
- **Small pilots or labs:** SQL Server Express only if its limits are explicitly accepted.

SQL Server 2022 is the conservative recommendation because it has operational maturity and active support. SQL Server 2025 may be preferable when maximizing support lifetime, but it requires compatibility evidence before production use.

Normalized Version and Edition

Scenario	Version	Edition
Stable enterprise production	SQL Server 2022	Standard or Enterprise depending on HA/DR and licensing
Controlled new adoption	SQL Server 2025	Standard or Enterprise
Development and QA	Same major as production	Developer
Demo or small pilot	SQL Server 2022/2025	Express, if limits are acceptable

Versions older than SQL Server 2022 are not recommended for new deployments.

PHP/Laravel Drivers

LoraTrack can connect to SQL Server through Laravel's `sqlsrv` connection.

Requirements:

- Microsoft ODBC Driver for SQL Server.
- Microsoft Drivers for PHP for SQL Server.
- PHP extensions:

- `pdo_sqlsrv`
- `sqlsrv`

The latest GA driver version observed during documentation was 5.13.1.

`.env` Configuration

```
DB_CONNECTION=sqlsrv
DB_HOST=sqlserver.example.com
DB_PORT=1433
DB_DATABASE=loratrack
DB_USERNAME=loratrack_app
DB_PASSWORD=CHANGE_ME_LONG_SECRET
```

For named instances, validate host and port format with the driver and network policy. Production should prefer an explicit port.

Database and User Creation

Initial example:

```
create database loratrack;
go

create login loratrack_app with password = 'CHANGE_ME_LONG_SECRET';
go

use loratrack;
go

create user loratrack_app for login loratrack_app;
go

alter role db_datareader add member loratrack_app;
alter role db_datawriter add member loratrack_app;
alter role db_ddladmin add member loratrack_app;
go
```

Notes:

- `db_ddladmin` supports Laravel migrations.
- In mature production, use a separate migration account and remove DDL privileges from the runtime account.

- Do not use `sa` for the application.
- Align credential custody and rotation with ISO/IEC 27001 and ISO/IEC 27002.

Collation and Unicode

LoraTrack stores names, descriptions, payloads, and metadata. Recommendations:

- Define collation in the project data design.
- Validate Unicode, accents, identifiers, and search behavior.
- Keep database, table, and column behavior consistent.

Backups

Minimum recommendation:

- daily full backup;
- differential or incremental backup based on RPO;
- transaction log backups if recovery model is Full;
- periodic restore testing;
- encrypted backups;
- contract-defined retention.

High Availability

For critical production, evaluate:

- SQL Server Always On Availability Groups;
- managed backup tooling;
- CPU, memory, IO, latency, and lock monitoring;
- separate data, log, and tempdb storage;
- index and statistics maintenance.

Initial Sizing

Sizing depends on:

- number of assets;
- uplink frequency;
- observations per uplink;
- raw payload retention;
- historical position volume;
- concurrent users.

Pilot starting point:

- 4 vCPU;
- 16 GB RAM;
- SSD;
- monitored storage for data, logs, and backups;
- weekly growth review.

Production sizing must be based on load testing and agreed retention.

Security

Recommendations:

- TLS for app-to-database connections when crossing networks.
- SQL Server not exposed publicly.
- Firewall limited to application servers.
- Least-privilege application account.
- Security and operational audit aligned with ISO/IEC 27001, ISO/IEC 27002, and ISO/IEC 20000-1.
- Encrypted backups.
- Transparent Data Encryption if required by policy.
- Do not log full connection strings.

Laravel Compatibility

Before certifying SQL Server as production database, run in staging:

```
php artisan migrate:fresh --seed
php artisan test
```

Then validate manually:

- login;
- organizations;
- connectors;
- TTI ingestion;
- queue jobs;
- maps;
- asset track;
- floor plan uploads;
- alerts;
- telemetry cleanup.

Reviewed Sources and Benchmarks

- Microsoft SQL Server downloads.
- Microsoft Learn: SQL Server 2025 features and Developer editions.
- Microsoft Learn: PHP drivers for SQL Server, GA version 5.13.1.
- Microsoft Learn: SQL Server support lifecycle.
- Microsoft Learn: SQL Server 2022 requirements.
- ISO/IEC 27001:2022.
- ISO/IEC 27002:2022.
- ISO/IEC 20000-1:2018.
- ISO 22301.
- CIS Microsoft SQL Server Benchmark.

Compliance Baseline and Benchmarks

Objective

Infrastructure decisions must not remain informal or open-ended. For enterprise LoraTrack deployments, database, server, operations, and evidence requirements should align with a concrete compliance baseline.

Recommended Baseline

Area	Reference	Usage in LoraTrack
Information security	ISO/IEC 27001:2022	ISMS governance, risk management, controls, continual improvement, and security evidence.
Security controls	ISO/IEC 27002:2022	Guidance for access control, cryptography, operations, incidents, suppliers, and information protection.
IT service management	ISO/IEC 20000-1:2018	Service operation, change, incident management, continuity, and service levels.
Business continuity	ISO 22301	RTO/RPO, impact analysis, continuity, and recovery.
SQL Server hardening	CIS Microsoft SQL Server Benchmark	Verifiable secure configuration for SQL Server instances and databases.
Platform lifecycle	Microsoft Lifecycle	Supported versions, end of support, patching, and upgrade planning.

Mandatory SQL Server Criteria

SQL Server selection must be defined by this baseline:

1. The SQL Server version must be within Microsoft Lifecycle support.
2. The edition must support availability, backup, audit, and growth requirements.
3. The instance must be hardened against the CIS Benchmark for its version.
4. Operation must produce evidence compatible with ISO/IEC 27001 and ISO/IEC 27002.
5. Backup, restore, and continuity must be documented against ISO 22301.
6. Incidents, changes, and recurring operations must be governed against ISO/IEC 20000-1.

Version Standard

For production:

- Conservative baseline: SQL Server 2022 Standard, latest approved Cumulative Update.
- Controlled new adoption: SQL Server 2025 Standard, only after staging validates migrations, PHP drivers, jobs, load, and CIS hardening.
- Versions older than SQL Server 2022 are not recommended for new deployments.

For development and QA:

- SQL Server Developer using the same major version as production.

For laboratory use:

- SQL Server Express only with explicit acceptance of its limits.

Minimum Evidence

- Exact SQL Server version and CU.
- Microsoft ODBC Driver version.
- Microsoft Drivers for PHP for SQL Server version.
- `php -m` output showing `pdo_sqlsrv` and `sqlsrv`.
- `.env` configuration without secrets.
- Migration execution evidence.
- Backup and restore evidence.
- SQL Server user and role matrix.
- CIS hardening checklist or scan result.
- Infrastructure change record.
- Approved RTO/RPO.
- Credential rotation procedure.

Sources

- ISO/IEC 27001:2022: <https://www.iso.org/standard/27001>
- ISO/IEC 27002:2022: <https://www.iso.org/standard/75652.html>
- ISO/IEC 20000-1:2018: <https://www.iso.org/standard/70636.html>
- ISO 22301: <https://www.iso.org/standard/75106.html>
- CIS Microsoft SQL Server Benchmark: https://www.cisecurity.org/benchmark/microsoft_sql_server
- Microsoft SQL Server downloads: <https://www.microsoft.com/en-us/sql-server/sql-server-downloads>
- Microsoft SQL Server Lifecycle: <https://learn.microsoft.com/en-us/lifecycle/products/?terms=SQL%20Server>

- Microsoft Drivers for PHP for SQL Server: <https://learn.microsoft.com/en-us/sql/connect/php/download-drivers-php-sql-server>

Documentation Limit

This baseline guides technical configuration and evidence. It does not state that LoraTrack, the supplier, or the customer environment is certified under ISO/IEC 27001, ISO/IEC 20000-1, or ISO 22301. Certification requires formal scope, organizational controls, audit, and approval by the corresponding certification body.

Operations, Monitoring, and Runbooks

Objective

Define the minimum tasks required to operate LoraTrack in an enterprise environment and diagnose common failures.

Required Processes

Scheduler

Run every minute:

```
php artisan schedule:run
```

The scheduler is the only required background executor. It processes Meraki batches and observations, TTI/MQTT telemetry, and requested catalog synchronizations without Laravel Queue. TTI and MQTT commands process at most three pending events per execution.

Scheduled tasks:

- `loratrack:evaluate-alerts` every 10 minutes.
- `loratrack:manage-telemetry-storage` hourly.
- `loratrack:prune-meraki-history` hourly.

MQTT Listener

When MQTT connectors are configured:

```
php artisan loratrack:mqtt-listen
```

The listener must be supervised and restarted on failure.

Operational Health View

Route:

```
/operations/health
```

Review:

- stuck telemetry;
- connector errors;
- private storage state;
- anchors by floor plan;
- recent audit records.

Logs

Laravel log:

```
storage/logs/laravel.log
```

Scheduler cron output example:

```
php artisan schedule:run >> storage/logs/schedule.log 2>&1
```

Do not enable tracing that prints credentials.

Runbook: Telemetry Stays Pending

1. Confirm scheduler execution:

```
php artisan schedule:run -v
```

2. Review recent events:

```
select id, connector_id, device_id, processing_status, processing_error,  
       observed_at, received_at, processed_at  
from telemetry_events  
order by received_at desc  
limit 20;
```

3. Check database connection limits if `max_user_connections` appears.
4. Confirm there is only one cron entry invoking the scheduler each minute.

Runbook: TTI Arrives but Asset Time Does Not Update

1. Check recent event identity and timestamps.
2. Check event processing status.
3. Check active asset-device assignment.

4. Check that the event has signal observations.
5. Check whether a position estimate was generated.

Useful SQL:

```
select id, device_id, observed_at, received_at, processing_status, processing_error
from telemetry_events
order by received_at desc
limit 10;
```

Runbook: Track View Shows Fewer Points Than Expected

The track view shows `position_estimates`, not raw `telemetry_events`.

Review:

```
select id, asset_id, telemetry_event_id, floor_plan_id, calculated_at, x, y
from position_estimates
where asset_id = '{asset_id}'
order by calculated_at desc;
```

If telemetry exists without a position, check:

- fewer than three valid MAC/RSSI observations;
- anchors not installed;
- inactive anchors;
- anchors on different floor plans;
- expired assignment;
- failed event;
- floor plan filter in the UI.

Runbook: Disabled Connector

A disabled connector does not process telemetry. Queued events are marked `ignored` when taken by the job.

To resume:

1. Activate connector.
2. Send a new payload or requeue events according to policy.
3. Confirm `last_activity_at` and `last_success_at`.

Runbook: Database Connection Limit Exhausted

Symptom:

```
SQLSTATE[42000] [1226] User has exceeded the max_user_connections resource
```

Actions:

- remove duplicate scheduler cron entries;
- review cron overlap;
- review persistent connections;
- increase database limits if load justifies it.

Monthly Maintenance Checklist

- review users and roles;
- review active connectors and tokens;
- test backup restoration;
- review `telemetry_events` growth;
- review failed jobs;
- validate SMTP alerts;
- review recurring log errors;
- run `composer audit` during a controlled window;
- update change documentation.

Escalation Data

For level 2/3 support collect:

- exact timestamp;
- organization;
- connector;
- device identifier;
- telemetry event ID;
- asset ID;
- position estimate ID when present;
- sanitized log excerpt;
- deployed release version;
- recent configuration changes.

Field Commissioning Guide

Objective

Define a baseline procedure for installing, validating, and calibrating LoraTrack in an industrial facility.

Suggested Roles

Role	Responsibility
Project lead	Scope, work windows, and acceptance.
OT/IT engineering	Network, access, servers, security, and integrations.
RF/IoT specialist	Devices, beacons, trackers, and gateways.
LoraTrack administrator	Organizations, users, connectors, and configuration.
Customer operations	Use case validation and acceptance criteria.

Prerequisites

- Current floor plan.
- Real dimensions in meters.
- Asset inventory.
- Device inventory.
- TTI, MQTT, or Meraki connectivity as required.
- LoraTrack environment access.
- Approved credentials and tokens.
- Authorized installation window.
- Defined success criteria.

Step 1: Environment

1. Confirm the deployed release version.
2. Confirm `APP_DEBUG=false`.
3. Confirm scheduler operation every minute.
4. Confirm scheduled telemetry processing.
5. Confirm SMTP if alerts are used.
6. Confirm backups.

7. Confirm basic monitoring.

Step 2: Organization and Users

1. Create organization.
2. Create administrator.
3. Configure branding if required.
4. Invite users.
5. Assign roles by responsibility.
6. Validate access with a non-admin user.

Step 3: Floor Plans

1. Create location.
2. Upload raster plan or preview image.
3. Register real width and height in meters.
4. Verify orientation.
5. Draw operational zones.
6. Store evidence of dimensions used.

Step 4: Fixed Devices

1. Register beacons or scanners.
2. Install physically.
3. Register coordinates on the floor plan.
4. Confirm correct type: `beacon` or `scanner`.
5. Confirm status `active`.
6. Register initial RSSI parameters: - reference RSSI; - path loss exponent.

Step 5: Connectors

TTI

1. Create TTI connector.
2. Generate a long token.
3. Activate connector.
4. Configure webhook in TTI.
5. Send a test payload.
6. Confirm event is `processed`.
7. Confirm observations in `signal_observations`.

MQTT

1. Create MQTT connector.
2. Validate host, port, TLS, and credentials.
3. Run listener.
4. Confirm message reception.

Catalog

1. Create connector.
2. Test connection.
3. Run synchronization.
4. Validate imported products and SKUs.

Step 6: Assets and Assignments

1. Create or import assets.
2. Register trackers or mobile beacons.
3. Assign device to asset.
4. Select strategy: - `fixed_beacons_mobile_tracker`; - `mobile_beacon_fixed_scanners`.
5. Validate start date.
6. Avoid duplicate active assignments.

Step 7: Position Validation

1. Place asset at a known point.
2. Wait for or force an uplink.
3. Confirm `telemetry_events`.
4. Confirm at least three valid observations.
5. Confirm `position_estimates`.
6. Compare calculated position with real point.
7. Record observed error.
8. Repeat in multiple points.

Step 8: Calibration

Use the floor plan calibration workbench:

1. Select strategy.
2. Enter real X/Y point.
3. Enter median RSSI per anchor.
4. Review RMSE and residuals.

5. Adjust reference RSSI and path loss exponent.
6. Apply only if the estimate improves.
7. Keep calibration history.

Step 9: Acceptance Criteria

Examples:

- telemetry processed under N seconds;
- percentage of uplinks with valid position;
- median error below N meters in pilot zone;
- map updates within expected interval;
- track view shows expected history;
- SMTP alerts reach recipients;
- dashboards load within target time;
- users only see their organization.

Step 10: Operational Handover

Deliver:

- customer-managed credentials;
- connector list;
- token rotation owners;
- device inventory;
- location and floor plan map;
- calibration parameters;
- runbooks;
- support procedure;
- accepted risks.

Commissioning Evidence

Store:

- date and time;
- release version;
- participants;
- floor plan used;
- test points;
- position results;
- detected errors;

- corrective actions;
- counterpart approval or sign-off.